

Génération de SPARQL sur Wikidata par few-shot prompting et sélection guidée par exécution : participation au défi WikiKGQA @ ISWC 2026*

Kenmegne Fokam Emeric Cyrille^{1,*}, Tela Tela Gilbert Ebenezer¹

¹ Université de Yaoundé I, Faculté des Sciences, BP 812 Yaoundé, Cameroun

Résumé

Nous décrivons notre participation aux deux pistes *with-mentions* (EN, ES) du défi WikiKGQA @ ISWC 2026, qui demande de traduire des questions en langue naturelle en requêtes SPARQL exécutables sur un sous-ensemble de Wikidata. Notre pipeline enchaîne quatre étapes : retrieval TF-IDF de trois exemples similaires, prompt *few-shot*, génération de cinq candidats à températures mixtes (un déterministe, quatre à 0,7), et sélection du premier candidat renvoyant un résultat non vide sur l'*endpoint*. Sur le développement (100 questions par langue), nous obtenons F1 macro = 0,634 (EN) et 0,607 (ES). Sur le test officiel Codabench (75 questions par langue), les scores sont 0,65 (EN) et 0,63 (ES). Deux catégories concentrent les échecs : les questions typées en « *which / qué* » qui reposent sur la chaîne P31/P279* incomplète sur l'*endpoint*, et les requêtes COUNT pour lesquelles le LLM omet fréquemment l'agrégat. L'ensemble n'utilise que des LLM ouverts (gpt-oss-120b, llama-3.3-70b) via le *free tier* de Groq Cloud.

Keywords

Knowledge Graph Question Answering, Text-to-SPARQL, Wikidata, Large Language Models, In-context Learning, QAWiki

1. Introduction

Le *Knowledge Graph Question Answering* (KGQA) permet d'interroger un graphe RDF en langue naturelle. L'intérêt est double : rendre un graphe ouvert comme Wikidata [1] accessible sans SPARQL, et produire une couche de raisonnement vérifiable — la requête générée peut être inspectée, corrigée et rejouée, contrairement aux réponses opaques d'un LLM. Deux difficultés dominent : la désambiguïsation des mentions vers les Q-identifiants, et la traduction de la structure logique de la question en un motif de graphe correct (propriétés, agrégats, filtres, jointures), le tout en contexte multilingue.

Le défi WikiKGQA @ ISWC 2026 fournit à cet effet 497 paires bilingues (EN / ES) question-SPARQL adossées au corpus QAWiki, un *endpoint* QLever [3] servant une vue simplifiée de Wikidata figée pour toute la durée du défi, et une évaluation *online* sur Codabench utilisant la F1 macro QALD [4]. Nous participons aux deux pistes *with-mentions*, où les Q-identifiants des entités mentionnées sont fournis avec la question — ce qui isole la difficulté de la génération de requête.

Contributions.

1. Un pipeline en quatre étapes (retrieval TF-IDF, *few-shot prompting*, génération multi-candidate, sélection guidée par exécution) entièrement construit sur des LLM ouverts.

. WikiKGQA Challenge @ ISWC 2026 : International Semantic Web Conference, October 2026, Nara, Japan

*. Papier de participation au défi WikiKGQA @ ISWC 2026, pistes **en-with-mentions** et **es-with-mentions**.

*. Auteur correspondant.

. ✉ emeric.kenmegne@facsciences-uy1.cm (K. F. E. Cyrille); gilbert.tela@facsciences-uy1.cm (T. T. G. Ebenezer)

.  0009-0000-0000-0001 (K. F. E. Cyrille); 0009-0000-0000-0002 (T. T. G. Ebenezer)

.  © 2026 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

2. Une évaluation dev et test des deux pistes *with-mentions*, avec F1 macro respectivement 0,634 / 0,607 et 0,65 / 0,63.
3. Une analyse d’erreurs structurée qui isole deux catégories résistantes : questions typées (*which* / *qué*) et requêtes COUNT.

2. Travaux connexes

KGQA et QALD. La série QALD [4], migrée sur Wikidata avec QALD-10, a fixé le protocole d’évaluation (F1 macro sur l’union des réponses). Les approches classiques par *parsers* supervisés ou *templates* obtiennent de bons résultats sur des distributions vues mais généralisent mal aux reformulations nouvelles.

Text-to-SPARQL. La génération seq2seq (T5, BART *fine-tuné*) produit une syntaxe correcte mais souffre d’IRI hallucinées : le modèle génère des propriétés plausibles qui n’existent pas dans le graphe cible. Les décodages à grammaire contrainte rétablissent la validité syntaxique mais pas la satisfaisabilité sémantique — c’est précisément la faille que notre sélection par exécution comble.

LLM et *in-context learning*. Depuis [7], plusieurs travaux [8, 9] montrent qu’un LLM instruit produit des requêtes SPARQL compétitives à partir de quelques exemples. KAPING [5] injecte le voisinage du graphe ; ChatKBQA [6] et DecAF [10] décomposent le processus. Nous héritons du principe *few-shot* guidé par retrieval, mais choisissons un retrieveur non appris (TF-IDF, [14]) par souci de reproductibilité.

Auto-cohérence. Wang *et al.* [11] popularisent le vote sur plusieurs échantillons ; Self-Refine [12] et Reflexion [13] y ajoutent une critique automatique. Notre sélection guidée par exécution s’inscrit dans cette famille en évacuant l’étape critique : l’*endpoint* lui-même joue le rôle d’oracle.

3. Tâche et données

Le défi demande, pour chaque question en langue naturelle, une requête SPARQL exécutable et l’ensemble des réponses attendues (Q-identifiants pour **SELECT**, valeur booléenne pour **ASK**, entier pour **COUNT**). Quatre pistes sont proposées : deux avec mentions d’entités fournies (EN et ES), deux sans. Nous couvrons les deux pistes *with-mentions*.

Le corpus QAWiki est distribué au format JSON dérivé du schéma QALD. Chaque item réunit un identifiant, la question bilingue, la liste des mentions (label + Q-id), la SPARQL de référence et les réponses (Fig. 1). La version 2026.2 contient 497 questions, réparties en 317 train / 100 dev / 80 test (75 effectivement scorées). Le graphe cible est servi par un *endpoint* QLever [3] spécifique au défi (backend.wikikgqa.skynet.coypu.org) ; contrairement à l’*endpoint* public de Wikidata, la vue est *simplifiée* et certaines hiérarchies (notamment P31/P279*) sont incomplètes. Une requête syntaxiquement parfaite peut donc renvoyer un résultat vide simplement parce que la propriété n’est pas matérialisée.

La métrique officielle est la F1 macro QALD [4] : pour chaque question, $F1_q = 2PR/(P + R)$ où P et R portent sur l’ensemble des réponses ; le score global est la moyenne $|Q|^{-1} \sum_q F1_q$. Nous rapportons aussi le nombre de questions parfaites ($F1_q = 1$) et nulles ($F1_q = 0$).

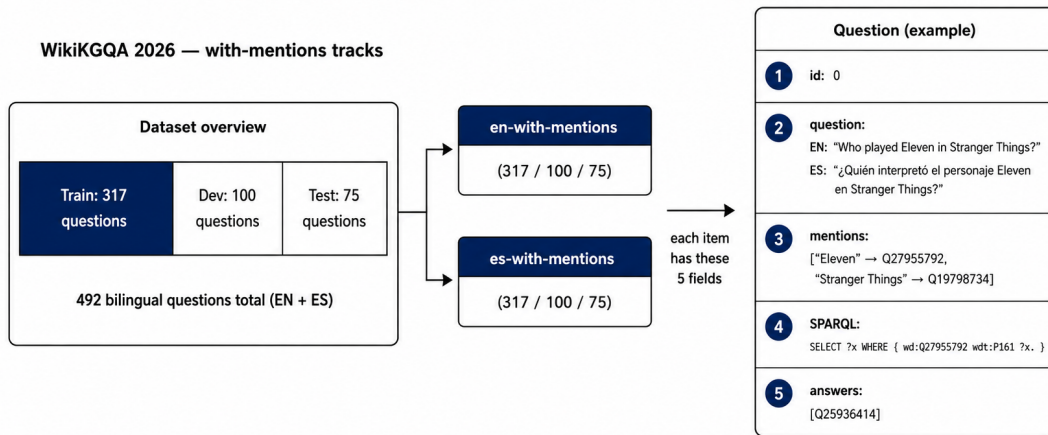


Figure 1: Format d'un item du corpus QAWiki : question bilingue, mentions d'entités avec Q-id, requête SPARQL de référence, réponses attendues.

4. Système proposé

4.1. Vue d'ensemble

La figure 2 synthétise notre pipeline, désigné P3. Entrée : une question et ses mentions. Sortie : une requête SPARQL et son ensemble de réponses. Le flux est linéaire, sans agent ni re-ranker appris.

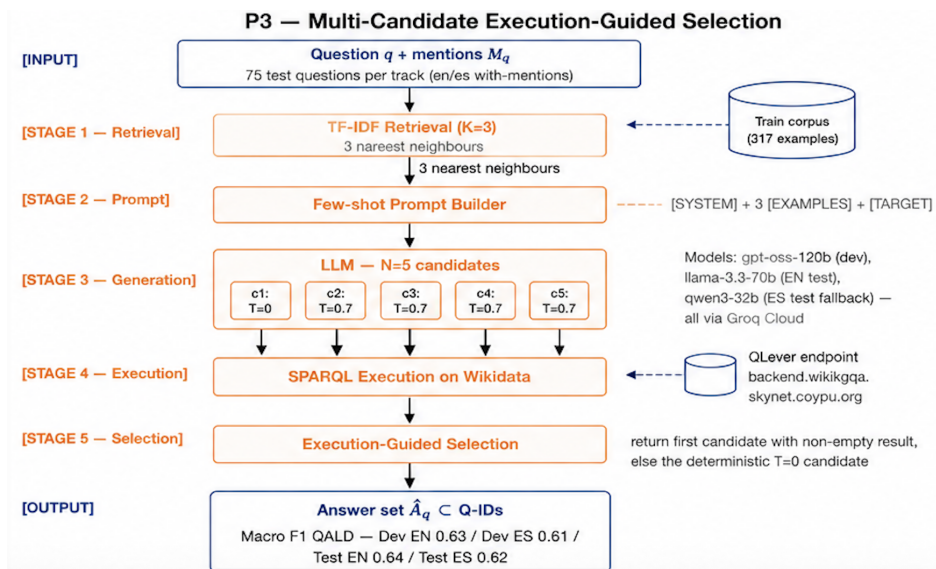


Figure 2: Pipeline P3. (1) Retrieval TF-IDF des trois voisins les plus proches; (2) prompt *few-shot*; (3) cinq candidats SPARQL (un à $T = 0$, quatre à $T = 0,7$); (4) exécution sur l'*endpoint*; (5) sélection du premier candidat non vide.

4.2. Retrieval et construction du prompt

Pour chaque question cible q , nous récupérons les $K = 3$ voisins les plus proches dans le train au sens du cosinus TF-IDF [14] : $\text{sim}(q, q') = v_q \cdot v_{q'} / (\|v_q\| \|v_{q'}\|)$, avec n -grammes de mots ($n \in \{1, 2\}$). Une heuristique légère infère le type cible (ASK, COUNT, SELECT) à partir des interrogatifs et privilégie les voisins de même type. L'entraînement du vectoriseur prend moins d'une seconde sur les 317 questions du train et ne dépend d'aucun modèle externe.

Le prompt enchaîne trois blocs (Fig. 3) : (i) une consigne système (« *You are an expert at generating SPARQL for Wikidata. Output ONLY the query inside a fenced code block. No PREFIX.* »), (ii) les K exemples formatés en triplets (question, mentions, SPARQL) chacun clos par un *fenced block* " `sparql ...` ", (iii) la question cible et ses mentions sous le même format, terminée par **SPARQL** : pour amorcer la génération. Les déclarations **PREFIX** sont volontairement omises et ajoutées automatiquement à l'exécution ; cela raccourcit le prompt et évite les préfixes hallucinés.

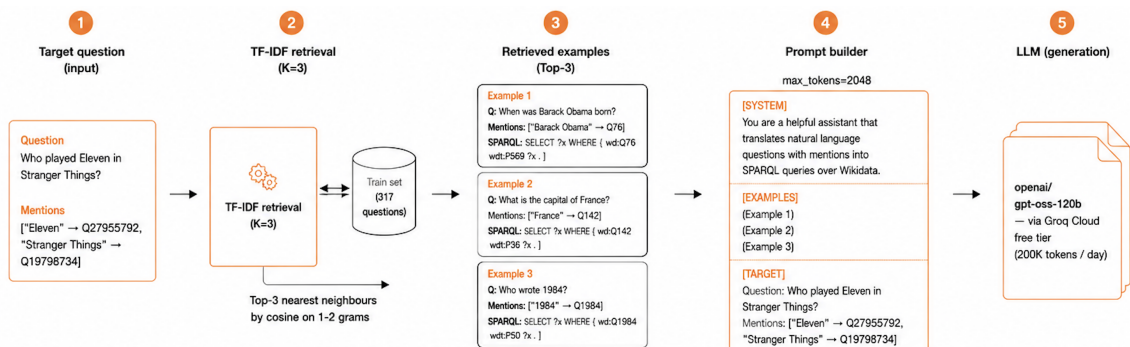


Figure 3: Construction du *prompt few-shot*. La question cible et ses mentions déclenchent un retrieval TF-IDF sur les 317 exemples du train ; les trois voisins retenus sont concaténés à la consigne système et à la cible avant envoi au LLM.

4.3. Génération multi-candidate et sélection par exécution

Nous interrogeons le LLM $N = 5$ fois pour la même question. Le premier appel est à $T = 0$ (pari le plus probable), les quatre suivants à $T = 0,7$ (diversification). Chaque candidat est extrait de son bloc de code, préfixé automatiquement par les **PREFIX** de Wikidata, puis soumis à l'*endpoint* via HTTP POST (*timeout* 30 s, *retry* adaptatif sur 429/5xx respectant le **Retry-After** de Groq). La règle de sélection est simple : nous renvoyons les réponses du premier candidat qui produit au moins une ligne. Si tous les candidats renvoient vide, nous conservons la sortie déterministe — ce qui couvre les rares cas où la réponse *gold* est effectivement vide (ASK négatif, SELECT avec MINUS).

L'intuition : le LLM connaît la structure publique de Wikidata mais ignore la vue simplifiée du défi. Une paraphrase entre candidats permet souvent de trouver un motif satisfaisable ; l'*endpoint* joue le rôle d'oracle en séparant les requêtes vides des exploitables.

4.4. Pré- et post-traitement

Trois garde-fous complètent le pipeline. (i) L'extracteur de mentions utilise la regex permissive ($Q\d{+}$) qui accepte les URI complètes (`http://.../Q42`) comme les Q-identifiants nus (`Q42`) — voir §6. (ii) Si le LLM omet le *fenced block*, un extracteur secondaire capture le premier motif ressemblant à une requête. (iii) En cas de *timeout* de l'*endpoint*, le candidat est marqué vide et le suivant est essayé (une exception non attrapée causait initialement des chutes silencieuses sur $\sim 3\%$ des questions).

Le pipeline ne différencie pas EN et ES au niveau du code : retrieveur, format du prompt et logique de sélection sont identiques. TF-IDF construit toutefois deux vocabulaires distincts (un par langue), et nous observons que la diversité inter-températures du LLM est plus faible en ES qu’en EN (§7).

5. Protocole expérimental

Nous utilisons les splits officiels (317 train / 100 dev / 75 test) et évaluons avec la F1 macro QALD (§3). Deux LLM ouverts servent nos expériences : `openai/gpt-oss-120b` [18] et `meta-llama/llama-3.3-70b-versatile`, accessibles via le *free tier* de Groq Cloud [17] (200 k et 100 k tokens/jour). Hyperparamètres : $K = 3$, $N = 5$, températures (0,0; 0,7; 0,7; 0,7; 0,7), `max_tokens=2048`, pause 6 s entre appels. Aucun *fine-tuning* ; le train sert uniquement à peupler l’index TF-IDF.

Le baseline principal est la variante *single-shot* de P3 ($N=1$, seul le candidat déterministe est produit), qui isole l’apport de la génération multi-candidate. Un *dev pass* complet prend 15–20 min sur un ordinateur portable sans GPU, la charge d’inférence étant reportée sur Groq. Coût monétaire nul.

Un pré-test a vérifié que notre implémentation locale du scorer reproduit exactement le score renvoyé par Codabench sur les soumissions dev, ce qui permet d’itérer localement avant chaque upload.

6. Résultats

6.1. Résultats dev et test

La table 1 rassemble les scores sur le développement et le test officiel Codabench.

Table 1

F1 macro sur dev (100 questions) et test (75 questions) des deux pistes *with-mentions*. *Single-shot* sert de borne inférieure (candidat $T = 0$ seul).

Piste	Méthode	F1 macro	Parfait	Nul
EN dev	Single-shot ($N=1$)	0,5524	52	40
EN dev	P3 (multi-candidate + exécution)	0,6342	60	31
ES dev	Single-shot ($N=1$)	0,5914	55	37
ES dev	P3 (multi-candidate + exécution)	0,6073	56	34
EN test	P3 (llama-3.3-70b)	0,65	—	—
ES test	P3 (gpt-oss-120b)	0,63	—	—

Le multi-candidate apporte +8,2 F1 en EN et seulement +1,6 en ES. L’asymétrie s’explique par la borne inférieure : en ES le candidat déterministe seul obtient déjà 0,59, ce qui laisse moins de marge de rattrapage. Les scores test (0,65 / 0,63) sont en parité étroite avec le dev, ce qui indique l’absence de surapprentissage à la distribution de développement.

6.2. Sensibilité du retrieveur

Le chevauchement Jaccard entre les voisins top-3 de TF-IDF et ceux de trois encodeurs neuronaux (`all-MiniLM-L6-v2`, `bge-small-en-v1.5`, `bge-m3` [16, 15]) tourne autour de 0,20. Un remplacement par un retrieveur neuronal reconstruirait donc massivement le contexte few-shot, ce qui constitue notre principale piste d’amélioration.

7. Analyse d’erreurs et discussion

Nous croisons les prédictions de P3 avec le type de requête généré, le mot interrogatif, le nombre de mentions et la longueur de la question.

7.1. Type de requête et mot interrogatif

La table 2 regroupe les deux vues. Les COUNT sous-performent (0,33 EN, 0,52 ES) : le mode d’échec récurrent est l’oubli de l’agrégat — le LLM produit `SELECT ?c WHERE { ... }` au lieu de `SELECT (COUNT(?c) AS ?n) WHERE { ... }`. À l’inverse, les ASK en espagnol atteignent 0,90 : les formulations oui/non ES (« *¿Es ... ?* », « *¿Tiene ... ?* ») sont plus contraintes qu’en EN.

Table 2

Performance par type de requête (bloc supérieur) et par mot interrogatif dominant (bloc inférieur), dev EN et ES.

Catégorie	EN			ES		
	<i>n</i>	F1	Nul	<i>n</i>	F1	Nul
<i>Par type de requête</i>						
SELECT	81	0,647	23	74	0,608	24
ASK	13	0,692	4	10	0,900	1
COUNT	6	0,333	4	13	0,520	6
<i>Par mot interrogatif dominant</i>						
which / qué	32	0,425	15	39	0,470	16
what / cuál	23	0,701	6	9	0,667	3
who / quién	13	0,822	1	13	0,800	2
when / cuándo	9	1,000	0	9	0,889	1
how-many / cuántos	5	0,200	4	10	0,400	6
yes-no	13	0,692	4	11	0,818	2

L’observation la plus nette est structurellement identique dans les deux langues : une seule catégorie d’interrogatifs concentre la moitié des nuls. En EN, les « *which* » (32 % du dev) totalisent 15 nuls sur 31 ; en ES, les « *qué* » (39 %) totalisent 16 nuls sur 34. Ces deux catégories partagent la même structure : filtrer un ensemble par leur type, ce qui passe par la chaîne `?x wdt:P31/wdt:P279* wd:Qtype`. Cette fermeture transitive est incomplète sur l’*endpoint* du défi ; le LLM génère la chaîne plausible mais l’exécution renvoie zéro ligne. À l’inverse, les questions courtes reposant sur une propriété directe (*when / cuándo*, *who / quién*) sont quasi-parfaites, car P569 (date de naissance) et P161 (distribution) sont bien matérialisées.

7.2. Effet du nombre de mentions et de la longueur, et rentabilité du multi-candidate

Deux gradients complémentaires apparaissent (Table 3) : (i) la F1 chute de 0,30 point en EN (0,23 en ES) entre les questions à 2 mentions et celles à 3+ ; (ii) elle chute de 0,16 en EN (0,19 en ES) entre les questions courtes (≤ 6 mots) et moyennes (7–10 mots). Ces effets convergent avec la catégorie « *which / qué* », plus longue et plus contrainte en moyenne.

La distribution des essais (Table 3, bloc inférieur) éclaire le mécanisme du multi-candidate. Trois régimes : (a) *immédiat* (essai 1) : 75 % des questions résolues au premier tir avec F1 moyenne $> 0,73$; (b) *rattrapage* (essais 2–4) : efficace en EN (7 gains à $F1 \geq 0,73$), marginal en ES ; (c) *épuisé* (essai 5, tous précédents vides) : rien récupéré (F1 moyenne 0,13 en EN, 0,00 en ES). Ces 16–18 questions par langue correspondent au profil homogène des P31/P279* et exigent un mécanisme différent du multi-candidate à températures.

Table 3

Trois vues complémentaires du dev : nombre de mentions (haut), longueur en mots (milieu), et essais nécessaires pour obtenir une réponse non vide (bas).

Catégorie	EN			ES		
	<i>n</i>	F1	Nul	<i>n</i>	F1	Nul
<i>Nombre de mentions</i>						
2	16	0,886	1	29	0,770	5
3+	84	0,586	30	71	0,541	29
<i>Longueur (mots)</i>						
0–6	39	0,732	8	30	0,736	6
7–10	54	0,571	20	58	0,543	24
11–14	7	0,571	3	12	0,596	4
<i>Essais nécessaires (F1 moyenne)</i>						
1 essai	75	0,737	—	74	0,799	—
2–4	7	0,870	—	5	0,304	—
5 (vides)	16	0,125	—	18	0,000	—

7.3. Discussion

Différences EN vs ES. Le candidat déterministe est plus souvent correct en ES (55 parfaits contre 52 en EN) ; le rattrapage multi-candidate y est en revanche nettement moins efficace, les températures 0,7 produisant des reformulations plus proches les unes des autres. La catégorie dominante d’échecs est isomorphe entre les deux langues (*which* $\leftrightarrow$ *qué*), pour la même raison technique.

Limites et pistes. Notre méthode ignore la structure réelle du graphe : elle génère à partir de la mémoire paramétrique du LLM et corrige a posteriori par exécution, ce qui produit une catégorie d’échecs prévisibles (chaînes transitives non matérialisées). TF-IDF rate par ailleurs les paraphrases (chevauchement Jaccard $\approx$ 0,2, §6). Trois directions ciblées : un retrieveur neuronal (bge-m3) pour les paraphrases ; un profilage léger de l’*endpoint* (`SELECT ?p (COUNT(?o) AS ?n)` par entité) pour fournir au LLM la liste des propriétés effectivement disponibles ; un tour de raffinement conditionnel à la Self-Refine [12] sur le régime *épuisé* de la Table 3.

8. Conclusion

Nous avons présenté notre participation aux deux pistes *with-mentions* du défi WikiKGQA @ ISWC 2026. Le pipeline enchaîne retrieval TF-IDF, prompt *few-shot*, génération de cinq candidats à températures mixtes et sélection guidée par l’exécution. Il atteint F1 macro = 0,634 (EN) et 0,607 (ES) sur le développement, 0,65 (EN) et 0,63 (ES) sur le test officiel Codabench, en utilisant uniquement des LLM ouverts. L’analyse d’erreurs isole deux catégories résistantes — questions typées *which* / *qué* et requêtes `COUNT` — et suggère trois pistes ciblées (retrieveur neuronal, profilage du *endpoint*, raffinement conditionnel).

Disponibilité. Le code, les données et les archives de soumission Codabench sont disponibles à <https://github.com/emeric-cyrille/wikikgqa-iswc2026>. Le corpus QAWiki (v2026.2) est distribué sous licence CC BY 4.0 sur Zenodo.

Références

- [1] D. Vrandečić, M. Krötzsch, Wikidata : A free collaborative knowledgebase, *Communications of the ACM* 57 (2014).
- [2] E. Prud'hommeaux, A. Seaborne, Sparql query language for rdf, in : *W3C Recommendation*, 2008.
- [3] H. Bast, B. Buchhold, Qlever : A query engine for efficient sparql+text search, in : *CIKM*, 2017.
- [4] R. Usbeck, A.-C. Ngonga Ngomo, B. Haarmann, A. Krithara, M. Röder, G. Napolitano, 7th open challenge on question answering over linked data (qald-7), in : *SemWebEval @ ESWC*, 2017.
- [5] J. Baek, A. F. Aji, A. Saffari, Kaping : Knowledge-augmented language model prompting for zero-shot knowledge graph question answering, in : *NLRSE Workshop @ ACL*, 2023.
- [6] H. Luo, H. E, Z. Tang, S. Peng, Y. Guo, W. Zhang, C. Ma, G. Dong, M. Song, W. Lin, Chatkbqa : A generate-then-retrieve framework for knowledge base question answering with fine-tuned large language models, in : *ACL Findings*, 2024.
- [7] T. Brown, B. Mann, N. Ryder, et al., Language models are few-shot learners, in : *NeurIPS*, 2020.
- [8] T. Li, X. Ma, A. Zhuang, Y. Gu, Y. Su, W. Chen, Few-shot in-context learning on knowledge base question answering, in : *ACL*, 2023.
- [9] J. Jiang, K. Zhou, Z. Dong, K. Ye, W. X. Zhao, J.-R. Wen, Structgpt : A general framework for large language model to reason over structured data, in : *EMNLP*, 2023.
- [10] D. Yu, S. Zhang, P. Ng, H. Zhu, A. H. Li, J. Wang, Y. Hu, W. Wang, Z. Wang, B. Xiang, Decaf : Joint decoding of answers and logical forms for question answering over knowledge bases, in : *ICLR*, 2023.
- [11] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. H. Chi, S. Narang, A. Chowdhery, D. Zhou, Self-consistency improves chain of thought reasoning in language models, in : *ICLR*, 2023.
- [12] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhume, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, P. Clark, Self-refine : Iterative refinement with self-feedback, in : *NeurIPS*, 2023.
- [13] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, S. Yao, Reflexion : Language agents with verbal reinforcement learning, in : *NeurIPS*, 2023.
- [14] G. Salton, C. Buckley, Term-weighting approaches in automatic text retrieval, *Information Processing & Management* 24 (1988).
- [15] N. Reimers, I. Gurevych, Sentence-bert : Sentence embeddings using siamese bert-networks, in : *EMNLP*, 2019.
- [16] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, Z. Liu, Bge m3-embedding : Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation, 2024. [arXiv:2402.03216](https://arxiv.org/abs/2402.03216).
- [17] Groq Inc., Groq cloud — inference platform for open-weight llms, 2024. <https://groq.com>.
- [18] OpenAI, gpt-oss-120b : Open-weight reasoning model, 2025. Open-weight release ; accessed via Groq Cloud.